

МОЛДАВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Математики и Информатики

Департамент Информатики

По предмету Дискретная математика: теория графов.

Лабораторная работа № 4

Тема: Вершинная раскраска графа.

Лектор: N. Cuciuc

Выполнил: D. Belov

Кишинев, 2026

1. Теория и определения

Задание: Минимальная правильная вершинная раскраска графа с использованием алгоритма последовательной окраски.

Вершинная k-раскраска графа - произвольная функция вида $f: X \rightarrow \{1, 2, \dots, k\}$ графа G.

Правильная раскраска - раскраска, при которой $f(x) \neq f(y)$ для любых смежных вершин x и y.

K-раскрашиваемый граф - граф, для которого существуют правильная k-раскраска.

Хроматическое число ($\chi(G)$) - Минимальное число k, при котором граф является k-раскрашиваемым.

Минимальная раскраска - правильная k-раскраска графа G при $k=\chi(G)$.

2. Описание алгоритма и принцип работы

Алгоритм последовательной окраски:

- Вершине, которая первая фигурирует в структуре, присваивается значение первого цвета.
- Переходим к обработке следующей вершины: присваиваем ей такой цвет, отличающийся от цвета смежных ей вершин.
- Повторяем до завершения раскраски графа.

Алгоритм программы:

- Алгоритм идет по вершинам в том порядке, в котором они были введены. От порядка обхода зависит итоговое количество цветов. Порядок ввода вершин определяет то, как программа видит граф, а значит может повлиять на результат).
- Мы создаем пустое множество `smj_colors` (использование `set` позволяет быстро проверять наличие цвета и исключает дубликаты).
- Проверяем каждую смежную вершину (`smj`) текущей. Если смежная вершина уже есть в словаре `result` (то есть он уже был раскрашен на предыдущих итерациях), мы записываем его цвет в список не используемых для текущей вершины.
- Мы начинаем проверку с самого первого цвета (индекс 0). Если цвет 0 уже занят кем-то из смежных, мы пробуем цвет 1. Если занят и он, то 2, и так далее. Цикл `while` остановится на самом маленьком числе, которого нет в наборе `smj_colors`.
- Как только мы нашли подходящее число, мы фиксируем его за текущей вершиной в словаре `result`

3. Примеры применения DFS и работы программы

Пример применения алгоритма

- Раскраска стран на атласах (вершинами выступают страны, а рёбрами - сухопутные границы).
- Классификация

Общий принцип: в качестве вершин мы берём объекты, которые необходимо классифицировать, через рёбра мы выражаем ограничения (пары вершин, которые мы не можем классифицировать одинаково), и при применении алгоритма минимальной правильной раскраски мы получаем классификацию вершин, где каждый отдельный цвет будет обозначать отдельную группу вершин, внутри которой не существует ограничений (нет рёбер).

Пример: классификация на парней и девушек. Пусть все студенты - вершины графа. Вместо того, чтоб прописывать каждому поле с полом (формирование классов), мы соединяем ребром студентов (вершины), если те имеют разный пол. Получаем граф, в котором каждая "девушка" смежна каждому "парню", при этом между собой ни одна пара вершин не смежна внутри одной "категории". При применении алгоритма раскраски граф со студентами окрасится таким образом, что парни будут одним цветом, а девушки - другим.

Пример работы программы

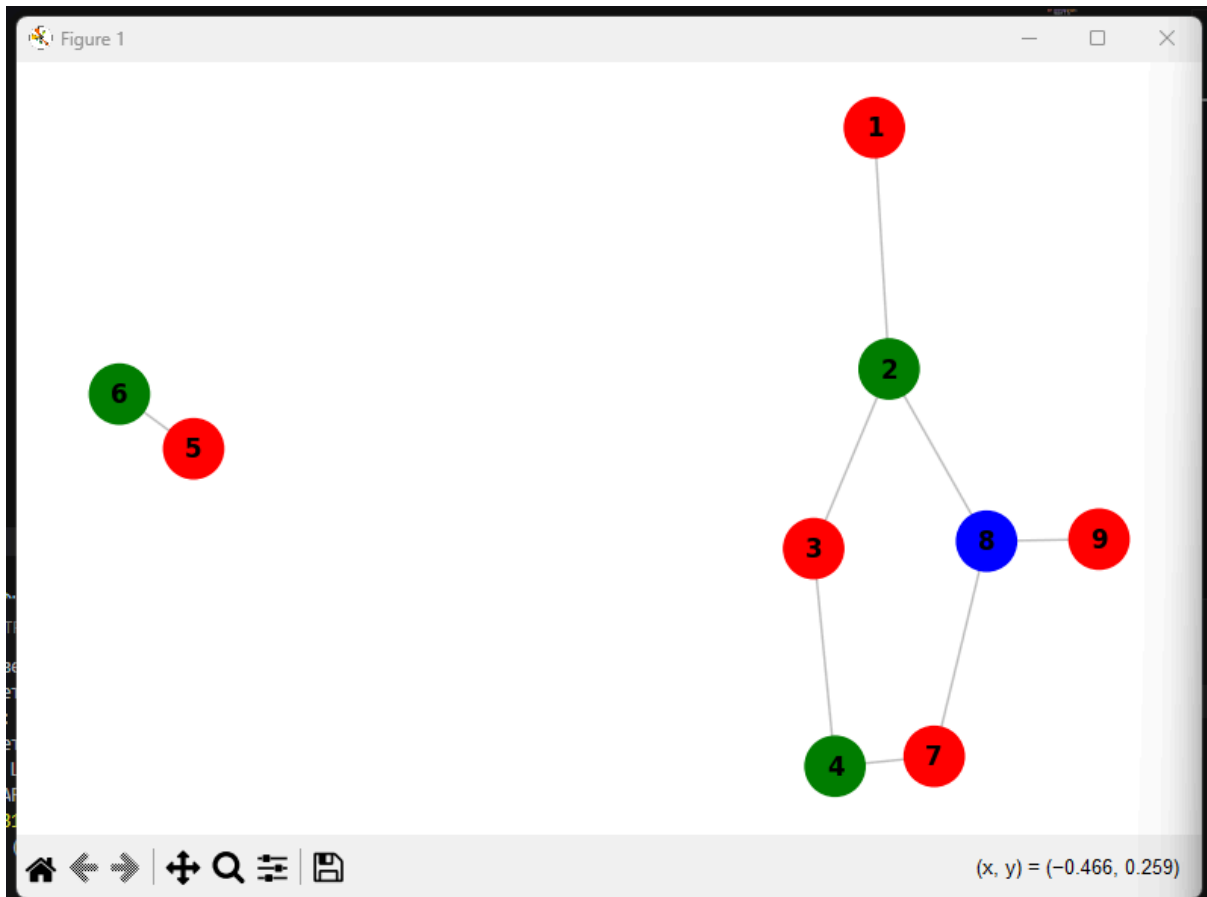
Введите ребра (например, 1 2). 'stop' для выхода:

```
> 1 2
> 2 3
> 3 4
> 5 6
> 7 8
> 8 9
> 4 7
> 2 8
> stop
```

Результат раскраски вершин:

```
Вершина 1: Цвет Красный
Вершина 2: Цвет Зеленый
Вершина 3: Цвет Красный
Вершина 4: Цвет Зеленый
Вершина 5: Цвет Красный
Вершина 6: Цвет Зеленый
Вершина 7: Цвет Красный
Вершина 8: Цвет Синий
Вершина 9: Цвет Красный
```

Визуализация:



Хроматическое число графа = 3

4. Выводы и оценка сложности

Временная сложность:

В худшем случае алгоритм делает $O(V + E)$ операций, где V - количество вершин, а E - ребер.

Пространственная сложность:

Список смежности: $O(V + E)$

Преимущества текущей реализации:

1. Использование динамических структур данных (dict, set) в Python позволяет работать с графами любого размера.
2. Устойчивость к вводу дубликатов ребер.
3. Автоматическая обработка изолированных вершин.
4. Нет никаких ограничений на названия вершин, однако из этого следует отсутствие сортировки при вводе.

5. Листинг программы

```
def coloring(graph):
    result = {}

    # Проходим по всем вершинам графа в порядке их следования
    for node in graph:
        # Находим цвета, которые уже заняты смежными вершинами
        smj_colors = set()
        for smj in graph[node]:
            if smj in result:
                smj_colors.add(result[smj])

        color = 0
        while color in smj_colors:
            color += 1

        result[node] = color

    return result

graph = {}
colors = {0: 'Красный', 1: 'Зеленый', 2: 'Синий', 3: 'Желтый', 4: 'Фиолетовый', 5:
'Оранжевый', 6: 'Голубой', 7: 'Коричневый', 8: 'Розовый', 9: 'Серый'}
print("Введите ребра (например, 1 2). 'stop' для выхода:")

while True:
    try:
        line = input("> ").strip()
        if line.lower() == 'stop': break
        if not line: continue

        parts = line.split()
        if len(parts) != 2:
            print("Пропуск некорректной строки")
            continue

        u, v = parts
```

```

    if u not in graph: graph[u] = set()
    if v not in graph: graph[v] = set()

    if u != v:
        graph[u].add(v)
        graph[v].add(u)

except EOFError:
    break

if not graph:
    print("\nГраф пуст.")
else:
    coloring = coloring(graph)
    print("Результат раскраски вершин:")
    for vertex, color in coloring.items():
        print(f"Вершина {vertex}: Цвет {colors.get(color, 'Не назначен')}")

import networkx as nx
import matplotlib.pyplot as plt

# Блок визуализации

if graph:
    G = nx.Graph()
    for node, neighbors in graph.items():
        G.add_node(node)
        for neighbor in neighbors:
            G.add_edge(node, neighbor)

# Словарь соответствия русских названий английским (для matplotlib)
color_map_translator = {
    'Красный': 'red', 'Зеленый': 'green', 'Синий': 'blue',
    'Желтый': 'yellow', 'Фиолетовый': 'purple', 'Оранжевый': 'orange',
    'Голубой': 'skyblue', 'Коричневый': 'brown', 'Розовый': 'pink', 'Серый': 'gray'
}

# Формируем список цветов, который точно совпадает с текстовым выводом
node_colors_for_plot = []
for node in G.nodes():
    color_id = coloring[node]
    russian_color = colors.get(color_id, 'Серый')
    english_color = color_map_translator.get(russian_color, 'gray')
    node_colors_for_plot.append(english_color)

```

```
plt.figure(figsize=(8, 6))
pos = nx.spring_layout(G)

nx.draw(
    G, pos,
    with_labels=True,
    node_color=node_colors_for_plot,
    node_size=800,
    font_weight='bold',
    edge_color='silver'
)

plt.title("Визуализация последовательной раскраски")
plt.show()
```